INFORME DE LABORATORIO 04

INFORMACIÓN BÁSICA					
ASIGNATURA:	**INTRODUCCIÓN AL DESARROLLO WEB**				
TITULO DE LA PRÁCTICA:	*JavaScript: Lógica y Manipulación del DOM*				
NÚMERO DE PRÁCTICA:	*04*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*II*
FECHA DE PRESENTACIÓN	*10/10/2026*	**HORA DE PRESENTACIÓN**	*hh/mm/ss*		
INTEGRANTE (s) *Christopher William Vásquez Vasquez*				**NOTA (0-20)**	
DOCENTE(s): **MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.**					

RESULTADOS Y PRUEBAS

1. EJERCICIO / PROBLEMA RESUELTO POR EL DOCENTE

Demostración de inserción dinámica en el DOM con manejo de eventos y localStorage:

```
// app.js - Módulo de Gestión
const form = document.querySelector('#todo-form');
const input = document.querySelector('#todo-input');
 const list = document.querySelector('#todo-list');
let tasks = JSON.parse(localStorage.getItem('tasks')) || [];
function renderTasks() {
    list.innerHTML = '';
    tasks.forEach((task, index) => {
       const li = document.createElement('li');
       li.className = 'list-group-item d-flex justify-content-between align-items-center';
       li.innerHTML = `
           <span>${task.text}</span>
           <button class="btn btn-danger btn-sm"
           onclick="deleteTask(${index})">Eliminar</button> `;
       list.appendChild(li);
       });
    }
form.addEventListener('submit', (e) => {
    e.preventDefault();
    const text = input.value.trim();
    if (!text) return;
    tasks.push({ text, completed: false });
    localStorage.setItem('tasks', JSON.stringify(tasks));
    input.value = '';
    renderTasks();
});
function deleteTask(index) {
    tasks.splice(index, 1);
    localStorage.setItem('tasks', JSON.stringify(tasks));
    renderTasks();
}

document.addEventListener('DOMContentLoaded', renderTasks);
```

2. EJERCICIOS / PROBLEMAS PROPUESTOS

Caso Práctico: Aplicación Interactiva de Gestión de Tareas Académicas

Desarrollar una aplicación interactiva "Gestor de Tareas Universitarias" en el navegador cliente sin refresco de página:

1. **Gestión del Estado y Estructuras de Datos:**
   - Mantener un arreglo de objetos JavaScript global para las tareas: [{ id, titulo, curso, fechaEntrega, completada }].
   - Utilizar métodos iterativos ES6+ (map, filter, reduce, find) para la manipulación del arreglo.

2. **Captura y Validación con JavaScript:**
   - Interceptar el evento submit del formulario de alta mediante preventDefault().
   - Validar mediante JS que ningún campo esté vacío y que la fecha de entrega sea posterior a la fecha actual. En caso de error, mostrar mensajes dinámicos en un div de alertas.

3. **Manipulación Dinámica del DOM:**
   - Renderizar la lista de tareas dinámicamente construyendo elementos DOM o usando template literals seguros.
   - Permitir alternar el estado de la tarea (completada/pendiente), modificando sus clases CSS dinámicamente (ej. tachado de texto).
   - Permitir la eliminación individual de una tarea del DOM y del arreglo.
   - Implementar un filtro visual (Todas / Pendientes / Completadas) manipulando el DOM.

4. **Persistencia de Datos:**
   - Sincronizar cada adición, cambio de estado o eliminación con localStorage.
   - Cargar y renderizar las tareas guardadas automáticamente al
   - desencadenarse el evento DOMContentLoaded.

Index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Gestor de Tareas Universitarias</title>
 <!-- Bootstrap 5 (CDN) -->
 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
 <link rel="stylesheet" href="css/style.css">
</head>
<body>
 <header class="bg-primary text-white text-center py-3">
  <h1 class="h3 mb-0">Gestor de Tareas Universitarias</h1>
 </header>
 <main class="container py-4">
  <!-- Formulario de alta -->
  <section class="card p-3 mb-4">
   <h2 class="h5">Nueva tarea</h2>
```

```
    <!-- Alertas dinámicas (se llenan desde JavaScript) -->
    <div id="alertas" aria-live="polite"></div>
    <form id="form-tarea" novalidate>
     <div class="row g-3">
      <div class="col-12 col-md-4">
       <label for="titulo" class="form-label">Título</label>
       <input type="text" id="titulo" class="form-control" placeholder="Ej. Informe Lab 04">
      </div>
      <div class="col-12 col-md-4">
       <label for="curso" class="form-label">Curso</label>
       <input type="text" id="curso" class="form-control" placeholder="Ej. Desarrollo Web">
      </div>
      <div class="col-12 col-md-4">
       <label for="fecha" class="form-label">Fecha de entrega</label>
       <input type="date" id="fecha" class="form-control">
      </div>
     </div>
     <button type="submit" class="btn btn-primary mt-3">Agregar tarea</button>
    </form>
   </section>
   <!-- Filtros y contador -->
   <section class="d-flex flex-wrap justify-content-between align-items-center mb-3 gap-2">
    <div class="btn-group" id="filtros" role="group" aria-label="Filtros de tareas">
     <button type="button" class="btn btn-outline-primary active" data-filtro="todas">Todas</button>
     <button type="button" class="btn btn-outline-primary" data-
filtro="pendientes">Pendientes</button>
     <button type="button" class="btn btn-outline-primary" data-
filtro="completadas">Completadas</button>
    </div>
    <p id="contador" class="mb-0 text-muted"></p>
   </section>
   <!-- Lista de tareas -->
   <section>
    <ul id="lista-tareas" class="list-group"></ul>
   </section>
  </main>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
  <script src="js/app.js"></script>
</body>
</html>
```

Style.css

```
/* css/style.css - Gestor de Tareas Universitarias */
* {
  box-sizing: border-box;
}
body {
  background-color: #f4f6f9;
}
/* Tarea completada: texto tachado y atenuado (se alterna desde JS) */
.tarea-completada strong {
  text-decoration: line-through;
  color: #6c757d;
}
.tarea-completada small {
  text-decoration: line-through;
}
.list-group-item {
  transition: background-color 0.2s;
}
.list-group-item:hover {
  background-color: #eef2f6;
}
```

App.js

```
// js/app.js - Gestor de Tareas Universitarias
// ===== 1. Referencias al DOM =====
const form = document.querySelector('#form-tarea');
const inputTitulo = document.querySelector('#titulo');
const inputCurso = document.querySelector('#curso');
const inputFecha = document.querySelector('#fecha');
const alertas = document.querySelector('#alertas');
const lista = document.querySelector('#lista-tareas');
const contador = document.querySelector('#contador');
const filtros = document.querySelector('#filtros');
// ===== 2. Estado global: arreglo de objetos =====
// [{ id, titulo, curso, fechaEntrega, completada }]
let tareas = [];
let filtroActual = 'todas';
// ===== 3. Persistencia con localStorage =====
const guardarTareas = () => {
  localStorage.setItem('tareas', JSON.stringify(tareas));
};
const cargarTareas = () => {
  try {
    return JSON.parse(localStorage.getItem('tareas')) || [];
  } catch (error) {
    // Si el JSON guardado está corrupto, se reinicia el estado
```

```
    console.error('Error al leer localStorage:', error);
    return [];
  }
};
// ===== 4. Utilidades =====
const hoyISO = () => {
  // Fecha local en formato YYYY-MM-DD (comparable como texto)
  const d = new Date();
  const mes = String(d.getMonth() + 1).padStart(2, '0');
  const dia = String(d.getDate()).padStart(2, '0');
  return `${d.getFullYear()}-${mes}-${dia}`;
};
const mostrarAlerta = (mensaje) => {
  alertas.innerHTML = '';
  const div = document.createElement('div');
  div.className = 'alert alert-danger py-2';
  div.textContent = mensaje; // textContent evita inyección de HTML
  alertas.appendChild(div);
};
const limpiarAlertas = () => {
  alertas.innerHTML = '';
};
// ===== 5. Renderizado dinámico (createElement, sin innerHTML con datos del usuario) =====
const filtrarTareas = () => {
  if (filtroActual === 'pendientes') return tareas.filter((t) => !t.completada);
  if (filtroActual === 'completadas') return tareas.filter((t) => t.completada);
  return tareas;
};
const crearItem = ({ id, titulo, curso, fechaEntrega, completada }) => {
  const li = document.createElement('li');
  li.className = 'list-group-item d-flex justify-content-between align-items-center gap-2';
  li.dataset.id = id;
  const info = document.createElement('div');
  info.className = completada ? 'tarea-completada' : '';
  const strong = document.createElement('strong');
  strong.textContent = titulo;
  const detalle = document.createElement('small');
  detalle.className = 'd-block text-muted';
  detalle.textContent = `${curso} · Entrega: ${fechaEntrega}`;
  info.append(strong, detalle);
  const acciones = document.createElement('div');
  acciones.className = 'd-flex gap-2';
  const btnEstado = document.createElement('button');
  btnEstado.className = 'btn btn-sm btn-success';
  btnEstado.dataset.accion = 'alternar';
  btnEstado.textContent = completada ? 'Reabrir' : 'Completar';
  const btnEliminar = document.createElement('button');
```

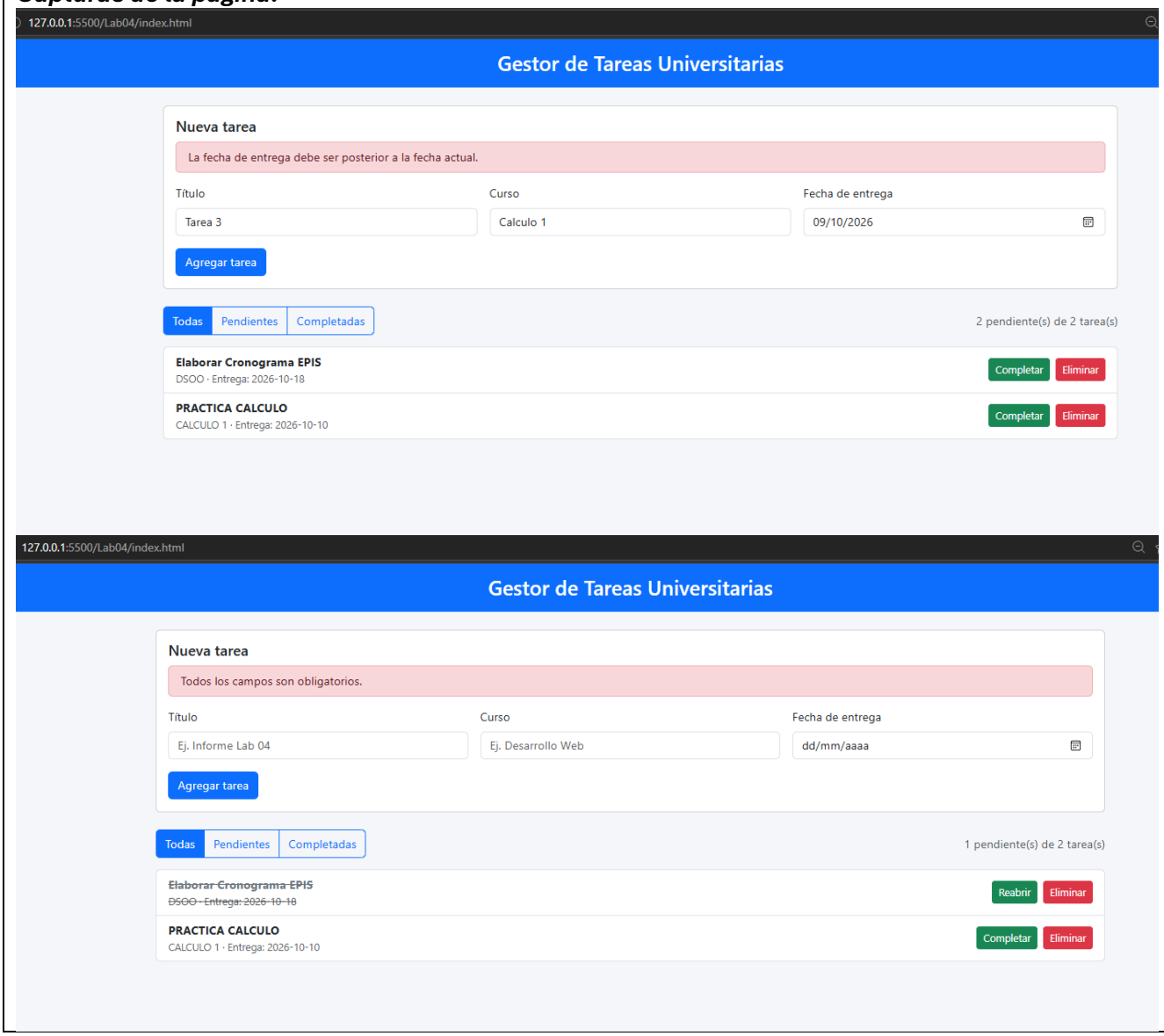
```
  btnEliminar.className = 'btn btn-sm btn-danger';
  btnEliminar.dataset.accion = 'eliminar';
  btnEliminar.textContent = 'Eliminar';
  acciones.append(btnEstado, btnEliminar);
  li.append(info, acciones);
  return li;
};
const renderTareas = () => {
  lista.innerHTML = '';
  const visibles = filtrarTareas();
  if (visibles.length === 0) {
    const vacio = document.createElement('li');
    vacio.className = 'list-group-item text-muted text-center';
    vacio.textContent = 'No hay tareas para mostrar.';
    lista.appendChild(vacio);
  } else {
    visibles.forEach((tarea) => lista.appendChild(crearItem(tarea)));
  }
  // reduce: cuenta las pendientes para el contador
  const pendientes = tareas.reduce((acc, t) => (t.completada ? acc : acc + 1), 0);
  contador.textContent = `${pendientes} pendiente(s) de ${tareas.length} tarea(s)`;
};
// ===== 6. Operaciones sobre el arreglo (map, filter, find) =====
const alternarTarea = (id) => {
  tareas = tareas.map((t) => (t.id === id ? { ...t, completada: !t.completada } : t));
  guardarTareas();
  renderTareas();
};
const eliminarTarea = (id) => {
  tareas = tareas.filter((t) => t.id !== id);
  guardarTareas();
  renderTareas();
};
// ===== 7. Eventos =====
// 7.1 Alta de tarea: interceptar submit + validación con JS
form.addEventListener('submit', (e) => {
  e.preventDefault();
  const titulo = inputTitulo.value.trim();
  const curso = inputCurso.value.trim();
  const fechaEntrega = inputFecha.value;
  if (!titulo || !curso || !fechaEntrega) {
    mostrarAlerta('Todos los campos son obligatorios.');
    return;
  }
  if (fechaEntrega <= hoyISO()) {
    mostrarAlerta('La fecha de entrega debe ser posterior a la fecha actual.');
    return;
```

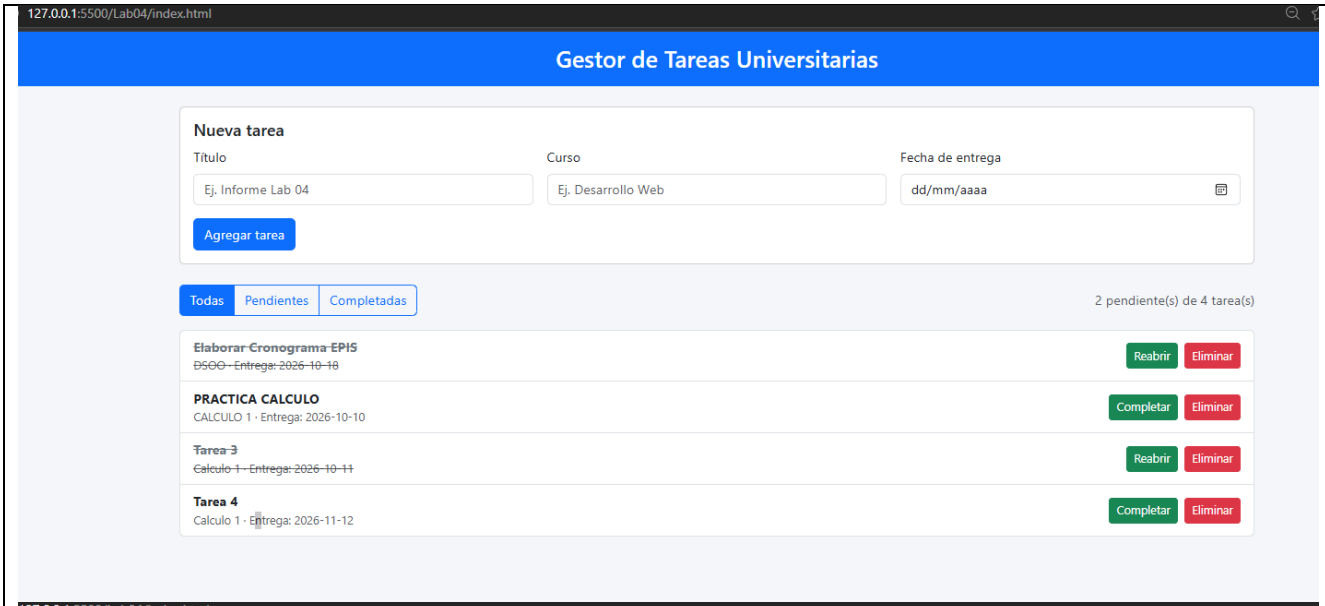
```
  }
  limpiarAlertas();
  tareas.push({ id: Date.now(), titulo, curso, fechaEntrega, completada: false });
  guardarTareas();
  form.reset();
  renderTareas();
});
// 7.2 Delegación de eventos: UN solo listener en la lista para todos los botones
lista.addEventListener('click', (e) => {
  const boton = e.target.closest('button[data-accion]');
  if (!boton) return;
  const id = Number(boton.closest('li').dataset.id);
  const tarea = tareas.find((t) => t.id === id); // find
  if (!tarea) return;

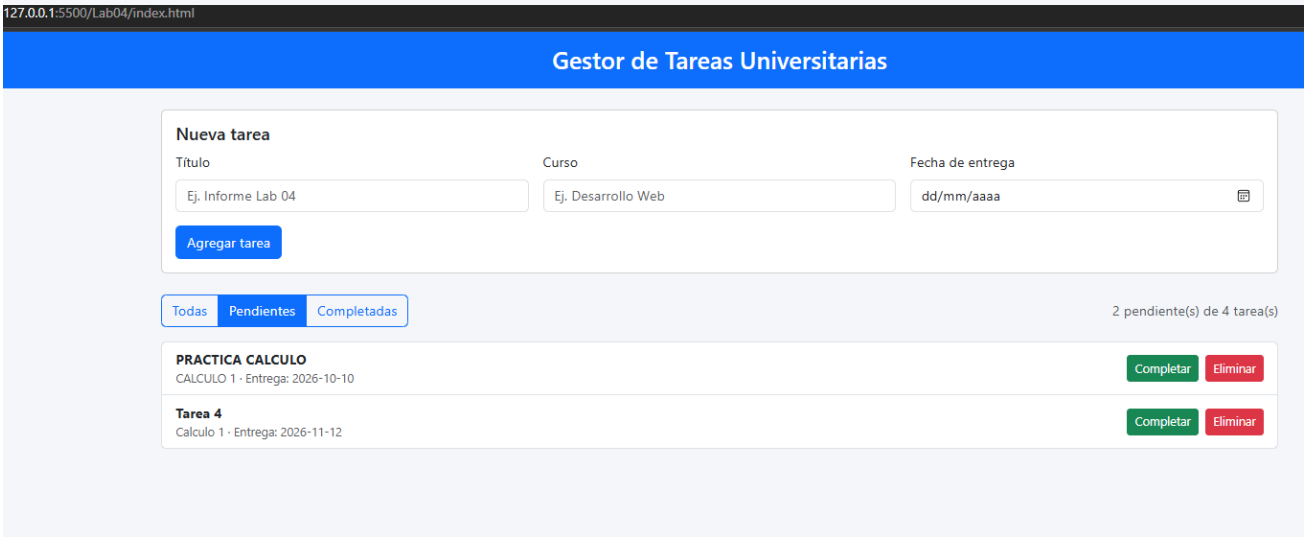
  if (boton.dataset.accion === 'alternar') alternarTarea(id);
  if (boton.dataset.accion === 'eliminar') eliminarTarea(id);
});
// 7.3 Filtros visuales (también por delegación)
filtros.addEventListener('click', (e) => {
  const boton = e.target.closest('button[data-filtro]');
  if (!boton) return;

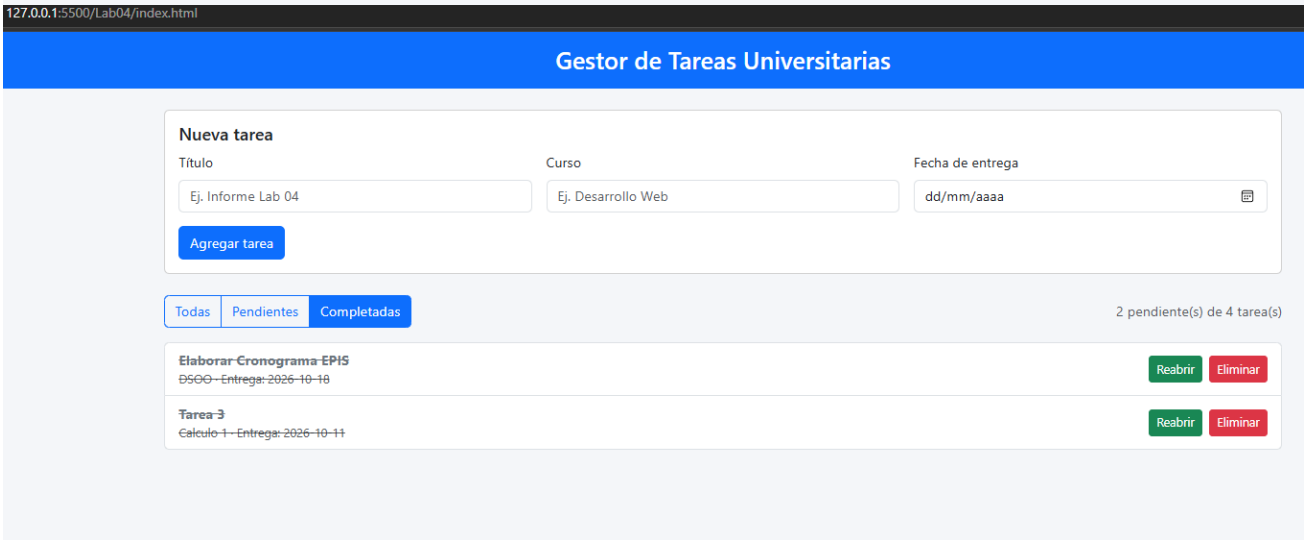
  filtroActual = boton.dataset.filtro;
  filtros.querySelectorAll('button').forEach((b) => b.classList.remove('active'));
  boton.classList.add('active');
  renderTareas();
});
// 7.4 Cargar y renderizar lo guardado al iniciar
document.addEventListener('DOMContentLoaded', () => {
  tareas = cargarTareas();
  renderTareas();
});
```

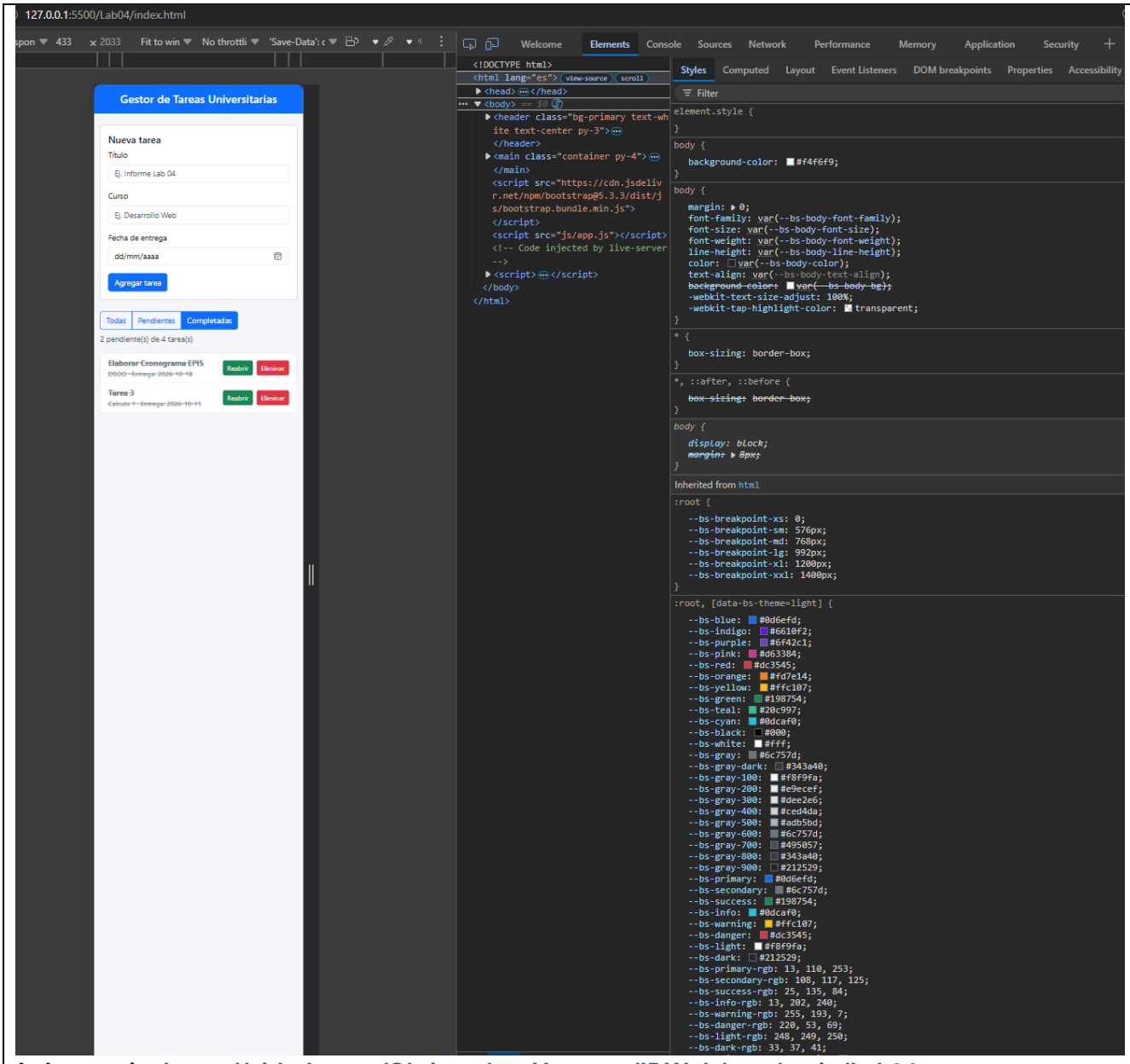
Capturas de la página:




127.0.0.1:5500/Lab04/index.html
Gestor de Tareas Universitarias
Nueva tarea
Título
Curso
Fecha de entrega
Ej. Informe Lab 04
Ej. Desarrollo Web
dd/mm/aaaa
Agregar tarea
Todas
Pendientes
Completadas
2 pendiente(s) de 4 tarea(s)
Elaborar Cronograma EPIS
DSOO · Entrega: 2026-10-18
Reabrir
Eliminar
PRACTICA CALCULO
CALCULO 1 · Entrega: 2026-10-10
Completar
Eliminar
Tarea 3
Calculo 1 · Entrega: 2026-10-11
Reabrir
Eliminar
Tarea 4
Calculo 1 · Entrega: 2026-11-12
Completar
Eliminar


127.0.0.1:5500/Lab04/index.html
Gestor de Tareas Universitarias
Nueva tarea
Título
Curso
Fecha de entrega
Ej. Informe Lab 04
Ej. Desarrollo Web
dd/mm/aaaa
Agregar tarea
Todas
Pendientes
Completadas
2 pendiente(s) de 4 tarea(s)
PRACTICA CALCULO
CALCULO 1 · Entrega: 2026-10-10
Completar
Eliminar
Tarea 4
Calculo 1 · Entrega: 2026-11-12
Completar
Eliminar


127.0.0.1:5500/Lab04/index.html
Gestor de Tareas Universitarias
Nueva tarea
Título
Curso
Fecha de entrega
Ej. Informe Lab 04
Ej. Desarrollo Web
dd/mm/aaaa
Agregar tarea
Todas
Pendientes
Completadas
2 pendiente(s) de 4 tarea(s)
Elaborar Cronograma EPIS
DSOO · Entrega: 2026-10-18
Reabrir
Eliminar
Tarea 3
Calculo 1 · Entrega: 2026-10-11
Reabrir
Eliminar



Laboratorio: ***https://github.com/Christopher-Vasquez/IDWeb/tree/main/Lab04***

3. CUESTIONARIO

1. **¿Cuál es la diferencia de comportamiento entre las palabras clave var, let y const respecto al alcance (scope) y al hoisting?**

   var tiene alcance de función; let y const, de bloque ({ }). Con var, la declaración se eleva (hoisting) y se inicializa en undefined, así que puede usarse antes de declararse. let y const también se elevan, pero quedan en la zona muerta temporal y lanzan ReferenceError si se usan antes.

   const exige valor inicial y no permite reasignación (el contenido de un objeto o arreglo sí puede cambiar). Se recomienda const por defecto, let si hay que reasignar y evitar var.

2. **Explique la técnica de Delegación de Eventos (Event Delegation) en el DOM y por qué es más eficiente que asignar listeners individuales a múltiples elementos.**
   Consiste en poner un solo listener en el elemento padre en lugar de uno por cada hijo, aprovechando la propagación (bubbling): el clic sube hasta el padre y ahí se identifica el origen con e.target. Es más eficiente porque usa menos memoria y funciona con elementos creados dinámicamente, sin tener que volver a asignar listeners al renderizar.

3. **¿Por qué es necesario utilizar JSON.stringify() al almacenar objetos complejos en localStorage y qué limitaciones tiene este almacenamiento en términos de seguridad?**
   localStorage solo guarda cadenas de texto; un objeto guardado directamente queda como "[object Object]". JSON.stringify() lo serializa y JSON.parse() lo reconstruye. En seguridad, localStorage no está cifrado, cualquier script del mismo origen puede leerlo (riesgo de XSS) y el usuario puede verlo y modificarlo desde DevTools. No debe guardar datos sensibles como contraseñas.

CONCLUSIONES

Los métodos de arreglos como map, filter, reduce y find permitieron manipular el estado de la aplicación de forma más clara y ordenada que con bucles tradicionales, reduciendo la cantidad de código necesario.
Construir los elementos del DOM con createElement y textContent, y manejar los clics mediante delegación de eventos, hizo que la lista de tareas respondiera correctamente incluso con elementos creados dinámicamente, además de evitar la inyección de HTML con datos del usuario.
Probar la aplicación recargando la página y revisando la pestaña Application de DevTools fue clave para confirmar que localStorage conservaba el estado completo de las tareas, y para recordar que este almacenamiento no es seguro para información sensible.
En general, descubrí que con JavaScript las páginas dejan de ser estáticas: ahora pueden reaccionar a lo que hace el usuario sin recargarse, algo que no era posible con los laboratorios anteriores hechos solo con HTML y CSS.

METODOLOGÍA DE TRABAJO

1. ***Análisis:*** *se revisaron los fundamentos de JavaScript ES6+, el DOM, los eventos y localStorage.*
2. ***Diseño:*** *se definió el estado como un arreglo de objetos { id, titulo, curso, fechaEntrega, completada } y se armó la interfaz con HTML y Bootstrap.*
3. ***Implementación:*** *se programó la lógica en app.js con map, filter, reduce y find; la validación del formulario con preventDefault(); el renderizado con createElement; y los eventos con delegación.*
4. ***Persistencia y pruebas:*** *se sincronizó cada cambio con localStorage, se cargaron los datos en DOMContentLoaded y se probaron en el navegador los casos de error, el cambio de estado, la eliminación y los filtros.*
5. ***Control de versiones:*** *se subió el trabajo a la carpeta Lab04/ del repositorio IDWeb con commits descriptivos.*